

PRÁCTICA 1 – FUERZA BRUTA Y BACKTRACKING

Contraseñas bajo ataque y bajo diseño: enumeración exhaustiva y construcción con poda

Modalidad: actividad individual o grupal, máximo 3 integrantes.

Porcentaje: 15%

Fecha de entrega: lunes 24 de agosto de 2026, hasta las 11:59 p.m.

Medio de entrega: enlace al repositorio GitHub publicado en InteractivaVirtual.

Lenguaje: C++ (obligatorio). Ver justificación en la Sección 4.

1. Contexto

Esta es la primera práctica del curso y la primera aproximación evaluada a los paradigmas de **Fuerza Bruta** y **Backtracking**. Ambos módulos comparten un mismo dominio, contraseñas y espacios de cadenas sobre un alfabeto, pero abordado desde dos roles opuestos:

- En el **Módulo FB** el estudiante actúa como analista de seguridad que, sin conocer una contraseña, debe determinar experimentalmente cuánto tiempo toma encontrarla probando exhaustivamente todas las combinaciones posibles.
- En el **Módulo BT** el estudiante actúa como diseñador de políticas de contraseñas que necesita generar –o contar– todas las contraseñas que cumplen una política de seguridad, sin generar primero todo el espacio de cadenas posibles y filtrar al final.

Esta simetría está diseñada para que el estudiante experimente en carne propia la diferencia entre "explorar todo sin ninguna estructura" (Fuerza Bruta) y "explorar de forma incremental descartando ramas tan pronto se sabe que no pueden llevar a una solución" (Backtracking), que es la distinción conceptual central de esta práctica.

Todo el trabajo se realiza sobre **datos sintéticos generados por el propio equipo**. En ningún momento de esta práctica se ataca, escanea o intenta vulnerar un sistema, cuenta, servicio o credencial real de un tercero. Esta condición es de cumplimiento obligatorio (ver Sección 17).

2. Objetivos de aprendizaje

Al finalizar esta práctica, el estudiante estará en capacidad de:

1. Identificar si un problema constituye una aplicación natural de Fuerza Bruta, de Backtracking, o de ninguno de los dos.
2. Modelar formalmente un espacio de búsqueda (alfabeto, longitud, estados parciales, estados objetivo).
3. Diseñar un algoritmo de enumeración exhaustiva y un algoritmo de backtracking con poda para instancias concretas de un problema.
4. Construir pseudocódigo correcto y consistente con la notación usada en el curso.
5. Implementar ambos algoritmos en C++17 de forma correcta y eficiente dentro de las restricciones del problema.
6. Analizar formalmente la complejidad temporal y espacial de cada algoritmo (mejor caso, caso promedio y peor caso cuando sea pertinente).

7. Diseñar experimentos que permitan contrastar el comportamiento teórico con el comportamiento empírico observado.
8. Cuantificar el efecto de la poda en Backtracking en términos de estados explorados y espacio de búsqueda reducido.
9. Interpretar y comunicar resultados experimentales mediante tablas y gráficas.
10. Documentar técnicamente una solución algorítmica siguiendo la estructura de un informe técnico.
11. Trabajar de forma colaborativa y verificable usando Git y GitHub.

3. Descripción general

La práctica está compuesta por dos módulos independientes en su implementación, pero conceptualmente complementarios:

	Módulo FB – Fuerza Bruta	Módulo BT – Backtracking
Rol del estudiante	Atacante que no conoce la contraseña	Diseñador de una política de contraseñas
Pregunta central	¿Cuánto cuesta encontrar una contraseña probando todo el espacio?	¿Cuántas / cuáles contraseñas cumplen la política, sin enumerar todo el espacio primero?
Técnica exigida	Enumeración sistemática + verificación por hash	Construcción incremental + poda por factibilidad
Comparación exigida	Fuerza bruta pura vs. ataque por diccionario reducido	Backtracking con poda vs. enumeración exhaustiva sin poda

Ambos módulos deben entregarse en un único repositorio de GitHub y en un único informe técnico (Secciones 11 y 12). El trabajo puede realizarse de forma individual o en equipos de hasta 3 integrantes (Sección 13).

4. Módulo FB – Fuerza Bruta: contexto y motivación

Un sistema que almacena contraseñas nunca guarda la contraseña en texto plano: guarda el resultado de aplicarle una función hash criptográfica (por ejemplo SHA-256). Cuando un atacante obtiene una base de datos de hashes filtrada, no puede "invertir" el hash – las funciones hash criptográficas están diseñadas para ser resistentes a preimagen. La única estrategia que garantiza recuperar la contraseña original, si esta pertenece a un espacio conocido y acotado, es probar sistemáticamente cada candidato posible del espacio, calcular su hash y compararlo contra el hash objetivo.

Esa estrategia –enumerar todo Σ^n sin usar ninguna propiedad adicional del problema para descartar candidatos– es la aplicación más directa y menos artificial del paradigma de **Fuerza Bruta** que existe en ciencias de la computación, y es también uno de los argumentos técnicos más usados para justificar por qué las políticas de longitud mínima y diversidad de caracteres son importantes: el costo de un ataque exhaustivo crece como $|\Sigma|^n$, y ese crecimiento es lo que este módulo pide medir y explicar, no solo enunciar.

Este módulo tiene un límite computacional evidente: para valores de n razonablemente pequeños (según el alfabeto elegido) el espacio de búsqueda ya es intratable en un computador personal. Parte del objetivo de este módulo es que el estudiante identifique experimentalmente, con sus propios datos, en qué punto ese crecimiento deja de ser manejable – el "muro exponencial" – y lo explique con el respaldo del análisis de complejidad correspondiente.

Alcance ético y técnico obligatorio: este módulo se realiza exclusivamente sobre contraseñas sintéticas generadas por el propio equipo mediante el procedimiento reproducible de la Sección 9.1, calculando su hash con una biblioteca criptográfica estándar (no se implementa SHA-256 desde cero). Queda terminantemente prohibido aplicar el programa desarrollado a cuentas, sistemas, bases de datos o credenciales reales, propias o de terceros. Este módulo no se debe presentar como, ni convertirse en, un ejercicio de intrusión.

No se espera, ni se debe, resolver aquí el diseño del algoritmo: esa es su tarea en la Sección 8.1.

5. Módulo BT – Backtracking: contexto y motivación

Como contraparte del Módulo FB, este módulo pone al estudiante en el rol de quien diseña o audita una política de contraseñas. Dada una política de complejidad por ejemplo, longitud fija, número mínimo de dígitos, número mínimo de mayúsculas, prohibición de caracteres consecutivos repetidos, interesa poder generar todas las contraseñas que la cumplen, o contarlas, sin necesariamente generar primero las $|Σ|^n$ cadenas posibles y filtrarlas al final (lo cual sería, de nuevo, fuerza bruta pura).

La alternativa es construir la contraseña **de forma incremental**, carácter por carácter, y verificar en cada paso si el prefijo parcial ya construido todavía puede llevar a una solución válida. Si en algún punto se determina que ninguna extensión del prefijo actual puede satisfacer la política (por ejemplo, porque ya no quedan posiciones suficientes para cumplir el mínimo de dígitos exigido), se abandona esa rama de inmediato sin explorar ninguna de sus extensiones: eso es **poda**, y esta construcción incremental con retroceso ante infactibilidad es exactamente la definición de **Backtracking**.

Este problema es un ejemplo típico de Satisfacción de Restricciones (CSP): las variables son los caracteres de cada posición, el dominio es el alfabeto, y las restricciones (mínimos de tipo de carácter, prohibiciones locales) son verificables de forma incremental sobre un prefijo – es precisamente esa verificabilidad incremental la que hace que la poda sea posible y efectiva.

Es importante que usted no infiera de este módulo que Backtracking cambia el **orden** de complejidad del problema: en el peor caso (una política casi vacía, que casi cualquier cadena satisface) Backtracking explora un árbol de tamaño comparable al de la enumeración exhaustiva, y la complejidad en el peor caso sigue siendo exponencial. Lo que la poda reduce, y lo que este módulo le pedirá cuantificar experimentalmente, es el número de nodos efectivamente visitados frente al tamaño teórico del espacio completo, para políticas con restricciones no triviales.

De nuevo: el diseño del algoritmo, la representación del estado y la condición de poda son su responsabilidad (Sección 8.2); esta sección solo explica por qué el problema es pertinente para el paradigma.

6. Fundamentación teórica

6.1 Fuerza Bruta

Su informe debe demostrar dominio de los siguientes conceptos, aplicados al problema del Módulo FB (no se exige, ni se permite, copiar definiciones de un libro sin conectarlas con su propia instancia del problema):

- **Definición del paradigma:** un algoritmo de fuerza bruta resuelve un problema generando y evaluando sistemáticamente todos los candidatos posibles del espacio de soluciones, sin usar conocimiento estructural del problema para reducir dicho espacio.

- **Espacio de búsqueda:** para un alfabeto Σ de tamaño $|\Sigma|$ y longitud fija n , el espacio de candidatos tiene tamaño $|\Sigma|^n$. Si se permiten longitudes de 1 a L , el espacio total es $\sum_{k=1}^L |\Sigma|^k$.
- **Enumeración exhaustiva:** debe existir un procedimiento sistemático que genere cada elemento del espacio exactamente una vez, sin omisiones ni repeticiones (esto debe justificarse formalmente, no solo implementarse).
- **Condiciones de aplicabilidad:** fuerza bruta es apropiada cuando el espacio de soluciones es finito y enumerable, no existe (o no se conoce) una propiedad estructural que permita descartar candidatos sin evaluarlos, y el tamaño de instancia es lo suficientemente pequeño para que el costo sea tolerable.
- **Complejidad temporal:** en función de $|\Sigma|$ y n , considerando el costo de evaluar cada candidato (en este problema, el costo de calcular un hash SHA-256, que se debe justificar como $O(1)$ respecto de n para longitudes de contraseña acotadas, o discutir por qué es razonable esa aproximación).
- **Complejidad espacial:** memoria requerida para generar y mantener un candidato a la vez, frente a la alternativa (incorrecta para este problema) de generar y almacenar todo el espacio antes de evaluarlo.
- **Ventajas:** simplicidad, garantía de encontrar la solución si existe dentro del espacio explorado, ausencia de supuestos sobre la estructura del problema.
- **Limitaciones:** crecimiento exponencial del costo respecto de n y $|\Sigma|$; impracticabilidad a partir de cierto tamaño de instancia.
- **Relación tamaño-costo:** debe presentar, con sus propios datos, cómo cada incremento de una unidad en n multiplica el costo por un factor de $|\Sigma|$, y contrastarlo con el crecimiento realmente medido en su experimentación (Sección 9).

6.2 Backtracking

- **Definición:** backtracking es una técnica de búsqueda exhaustiva sobre un árbol de estados parciales, donde las soluciones se construyen incrementalmente y cada rama se abandona ("se poda") tan pronto se determina que no puede llevar a una solución válida.
- **Relación con la búsqueda exhaustiva:** backtracking sigue siendo, en el peor caso, una técnica de exploración completa del espacio de soluciones; su diferencia con fuerza bruta no está en la garantía de completitud (ambas la tienen) sino en el **orden** de exploración y en el uso de información parcial para evitar explorar subárboles completos.
- **Construcción incremental de soluciones:** una solución de tamaño n se construye extendiendo, paso a paso, una solución parcial de tamaño $k < n$.
- **Representación del estado:** debe definir formalmente qué información describe un estado parcial (por ejemplo: el prefijo construido hasta el momento, y los contadores necesarios para evaluar las restricciones de la política).
- **Condiciones de factibilidad:** una función que determina si un estado parcial todavía puede, en principio, extenderse hasta una solución completa válida. Esta función es la que hace posible la poda.
- **Poda:** el acto de no generar los descendientes de un estado que la función de factibilidad determina como no viable.
- **Árbol de búsqueda (o de estados):** representación del espacio de exploración donde la raíz es el estado vacío, cada nivel corresponde a fijar un carácter adicional, y las hojas son cadenas completas de longitud n .
- **Diferencia entre explorar todo el espacio y podar:** debe cuantificar, no solo describir, cuántos nodos del árbol completo (tamaño teórico $\sum_{k=0}^n |\Sigma|^k = |\Sigma|^{n+1} - 1$) fueron efectivamente visitados por su implementación con poda.
- **Complejidad temporal y espacial:** el peor caso de backtracking para este problema conserva la cota exponencial de fuerza bruta (una política sin restricciones

efectivas no poda nada); la complejidad espacial está gobernada por la profundidad del árbol ($O(n)$ para la pila de recursión) más el costo de almacenar las soluciones encontradas si se piden explícitamente.

- **Mejor caso, caso promedio y peor caso:** el mejor caso ocurre cuando la política es tan restrictiva que casi cualquier prefijo se descarta tempranamente (poda casi inmediata); el peor caso ocurre cuando la política es casi vacía; el caso promedio depende de la distribución de restricciones y debe discutirse con base en los datos de su propia experimentación.

7. Actividades prácticas

7.1 Módulo FB — Fuerza Bruta

Debe desarrollar, como mínimo, las siguientes doce actividades:

1. **Modelamiento del problema.** Defina formalmente el espacio de búsqueda: alfabeto(s) a usar, rango de longitudes, y la función de verificación (comparación de hashes).
2. **Definición de entradas y salidas.** Entrada: un hash objetivo (cadena hexadecimal SHA-256), el alfabeto y la longitud máxima a explorar. Salida: la contraseña en texto plano si se encuentra dentro del espacio definido, o un reporte explícito de "no encontrado dentro del espacio" si se agota sin éxito.
3. **Definición de las restricciones.** Explícite los alfabetos y longitudes que va a soportar (ver Sección 9.1) y los límites de tiempo/tamaño que hacen tratable el experimento en un computador personal.
4. **Diseño del algoritmo.** Diseñe usted mismo el procedimiento de enumeración sistemática del espacio Σ^n (para n creciente) que evalúa cada candidato exactamente una vez.
5. **Pseudocódigo.** Escriba el pseudocódigo de su algoritmo siguiendo la notación empleada en el curso.
6. **Implementación** en C++17.
7. **Construcción de casos de prueba** siguiendo el procedimiento reproducible de la Sección 9.1.
8. **Ejecución con diferentes tamaños de entrada** (longitudes y alfabetos, según la Sección 9.1).
9. **Medición del tiempo de ejecución** para cada configuración.
10. **Análisis de resultados** experimentales.
11. **Análisis de complejidad**, contrastando la cota teórica $|\Sigma|^n$ con el crecimiento medido.
12. **Conclusiones** del módulo.

7.2 Módulo BT — Backtracking

Debe desarrollar, como mínimo, las siguientes doce actividades:

1. **Modelamiento del problema.** Defina el estado parcial (prefijo de longitud k), el estado inicial (cadena vacía), los estados terminales (cadenas de longitud n) y la política de contraseñas que va a implementar (ver Sección 9.2 para los parámetros específicos de su instancia).
2. **Definición de entradas y salidas.** Entrada: alfabeto, longitud n y los parámetros de la política asignada a su equipo. Salida: según la instancia solicitada, el conjunto completo de contraseñas válidas, o su conteo, o una contraseña válida arbitraria (especifique claramente cuál calcula cada ejecución).
3. **Definición de las restricciones.** Formalice cada regla de la política asignada a su equipo como una condición verificable sobre un prefijo parcial.

4. **Diseño del algoritmo.** Diseñe usted mismo la función de factibilidad y el procedimiento de construcción incremental con retroceso.
5. **Pseudocódigo.**
6. **Implementación** en C++17.
7. **Construcción de casos de prueba** siguiendo el procedimiento reproducible de la Sección 9.2.
8. **Ejecución con diferentes tamaños** (longitud n y distintas variantes de política, ver Sección 9.2).
9. **Medición de tiempo de ejecución, número de estados explorados y número de estados podados.**
10. **Análisis de resultados.**
11. **Análisis de complejidad**, discutiendo mejor caso, peor caso y comportamiento observado.
12. **Conclusiones** del módulo.

8. Experimentación y comparación algorítmica

Para cada módulo debe presentar, como mínimo:

- Una **tabla de tiempos** de ejecución para cada tamaño de entrada evaluado (mínimo 5 configuraciones distintas por módulo).
- Al menos **una gráfica** (tiempo de ejecución vs. tamaño de entrada) por módulo.
- Una comparación explícita entre el **crecimiento teórico** (según el análisis de complejidad de la Sección 6) y el **crecimiento empírico** observado, explicando cualquier desviación relevante (efectos de caché, sobrecarga de E/S, precisión del reloj, etc.).
- La identificación explícita del **punto a partir del cual el crecimiento del costo deja de ser manejable** en su equipo de cómputo, con la evidencia (tiempos medidos) que lo sustenta.

8.1 Comparación específica – Módulo FB: fuerza bruta pura vs. ataque por diccionario

Además de lo anterior, el Módulo FB exige comparar su implementación de fuerza bruta pura contra un **ataque por diccionario** sobre la lista sintética ``resources/diccionario.txt`` (provista por el curso, ver Sección 9.1). Debe reportar, para el mismo conjunto de hashes objetivo: tiempo de ejecución de cada estrategia, tasa de éxito de cada estrategia (el diccionario no garantiza encontrar la contraseña si esta no está en la lista) y una discusión explícita de por qué el ataque por diccionario no es exhaustivo aunque en la práctica suele ser mucho más rápido.

8.2 Comparación específica – Módulo BT: con poda vs. sin poda

El Módulo BT exige comparar su implementación con poda contra una versión de **exploración exhaustiva sin poda** (recorre el árbol completo hasta las hojas y filtra al final). Para el mismo conjunto de instancias debe reportar, en una tabla: número de nodos generados por la versión sin poda, número de nodos efectivamente visitados por la versión con poda, porcentaje de reducción del espacio de búsqueda, número de soluciones encontradas (debe coincidir entre ambas versiones, como verificación de correctitud) y tiempo de ejecución de cada versión.

9. Casos de prueba e instancias reproducibles

Para que el docente pueda verificar objetivamente los resultados y detectar copias entre equipos, cada equipo trabaja sobre una **instancia parcialmente propia**,

derivada de una semilla calculada a partir de los apellidos de sus integrantes, además de instancias de referencia comunes a todo el curso.

9.1 Instancias del Módulo FB

- **Alfabetos:** $A1 = \{a...z\}$ (26 símbolos); $A2 = \{a...z, 0...9\}$ (36 símbolos).
- **Longitudes a evaluar:** $n \in \{3, 4, 5, 6\}$ para $A1$; $n \in \{3, 4, 5\}$ para $A2$ (más allá de estos valores el espacio deja de ser tratable en un computador personal; si su equipo cuenta con más cómputo puede extender el rango y reportarlo como valor agregado).
- **Instancia de referencia (común a todo el curso, para validar que su implementación es correcta antes de continuar):** contraseña "abc12" sobre $A2$, $n = 5$. Hash SHA-256 (verificado con ``sha256sum``): ``8d51feb34e3e69f6fa6dffc577e2c60490cf9a7fcd835f9f6af1505b71d74773``.
- **Instancias del equipo (privadas, para el informe):** ordene alfabéticamente los apellidos de los integrantes, concatene todos los apellidos en minúsculas y sin espacios ni tildes, y calcule $\text{semilla} = (\text{suma de los códigos ASCII de todos los caracteres de la cadena resultante}) \bmod 100000$. Con esa semilla, genere 5 contraseñas objetivo usando el generador congruencial lineal $x_0 = \text{semilla}$, $x_{i+1} = (1103515245 \cdot x_i + 12345) \bmod 2^{31}$, tomando el carácter i -ésimo de cada contraseña como $\text{alfabeto}[x_i \bmod |\text{alfabeto}|]$. Use longitudes 4, 4, 5, 5, 6 (en ese orden) para las 5 contraseñas, alternando entre $A1$ y $A2$ como se indique en el enunciado publicado en InteractivaVirtual. Calcule el SHA-256 de cada una de las 5 contraseñas resultantes: esos 5 hashes son las instancias que su programa debe resolver.
- **Diccionario sintético:** ``resources/diccionario.txt``, con 500 candidatos (contraseñas comunes ficticias generadas para el curso), provisto en el repositorio de la práctica. No debe contener contraseñas reales filtradas de ningún sistema.
- **Métricas a registrar por instancia:** tiempo de ejecución (ms), candidatos evaluados, resultado (encontrada / no encontrada), y para la comparación con diccionario, si la contraseña objetivo pertenece o no a ``diccionario.txt``.

9.2 Instancias del Módulo BT

- **Alfabeto base:** minúsculas, mayúsculas, dígitos y el conjunto de símbolos `{!, @, #, $, %}` (69 símbolos en total).
- **Longitud fija de la política:** $n = 8$ para todas las instancias.
- **Parámetros de la política del equipo (derivados de la misma semilla de la Sección 9.1):** $\text{minLower} = 2 + (\text{semilla} \bmod 3)$; $\text{minUpper} = 1 + (\text{semilla} \bmod 2)$; $\text{minDigit} = 1 + (\text{semilla} \bmod 3)$; $\text{minSymbol} = 1$; prohibición de dos caracteres idénticos consecutivos (aplica a todas las instancias). Verifique que $\text{minLower} + \text{minUpper} + \text{minDigit} + \text{minSymbol} \leq n = 8$; si no se cumple, reduzca minLower en la cantidad necesaria y repórtelo en el informe.
- **Variantes de dificultad a evaluar (mínimo 5), variando la restrictividad de la política:** (i) política completa del equipo, $n = 8$; (ii) misma política, $n = 6$; (iii) misma política, $n = 10$; (iv) política relajada (solo $\text{minLower} = 1$, sin las demás restricciones), $n = 8$; (v) política sin ninguna restricción de composición (equivalente a enumeración exhaustiva), $n = 6$, usada exclusivamente para calibrar el caso de "poda nula".
- **Instancia de referencia común:** política $\text{minLower}=2$, $\text{minUpper}=1$, $\text{minDigit}=1$, $\text{minSymbol}=1$, sin repetidos consecutivos, $n = 6$, alfabeto base completo – use esta instancia para validar su implementación antes de ejecutar sobre las instancias propias del equipo.
- **Métricas a registrar por instancia:** tiempo de ejecución, nodos generados (sin poda), nodos visitados (con poda), número de soluciones encontradas, porcentaje de reducción del espacio de búsqueda.

Ambos procedimientos de generación de semilla son deterministas: dos ejecuciones del mismo equipo, o una verificación del docente con los mismos apellidos, deben producir exactamente los mismos valores. El enunciado publicado en InteractivaVirtual incluirá una pequeña utilidad de referencia (``resources/verificar_semilla.cpp``) para que cada equipo confirme su propia semilla antes de generar resultados.

10. Lenguaje de programación

Se mantiene **C++17**, en continuidad con la práctica integradora de referencia, por las siguientes razones: (i) permite medir tiempos de ejecución con ``std::chrono`` con la precisión y el bajo overhead necesarios para observar el crecimiento exponencial en instancias pequeñas, donde un lenguaje interpretado introduce ruido de medición comparable a las diferencias que se busca observar; (ii) el estándar incluye estructuras suficientes (``std::string``, recursión, ``<chrono>``) sin requerir bibliotecas externas complejas; (iii) mantiene coherencia con las prácticas previas y posteriores del curso, evitando que el estudiante deba aprender un lenguaje nuevo para una práctica cuyo objetivo es algorítmico, no lingüístico; (iv) favorece la reproducibilidad, al fijar un único compilador y estándar (``g++ -std=c++17 -O2``) para todos los equipos. Para el cálculo de SHA-256 se autoriza el uso de una biblioteca de cabecera única de dominio público (por ejemplo, ``picosha2.h``), que debe incluirse en ``src/third_party/`` con su licencia; no se debe implementar el algoritmo SHA-256 desde cero (eso desviaría el esfuerzo del objetivo algorítmico de la práctica) ni tampoco usarlo como una "caja negra" sin poder explicar, en términos generales, qué garantía de seguridad ofrece.

11. Estructura del repositorio en GitHub

Nombre del repositorio: ``ADA_P1_Apellido1_Apellido2_Apellido3`` (en orden alfabético; use solo sus apellidos si trabaja de forma individual o en pareja).

Ruta	Contenido
README.md	Instrucciones de compilación, ejecución y reproducción de experimentos; integrantes; estructura del proyecto.
src/	Código fuente de ambos módulos (<code>fb_*.cpp/hpp</code> , <code>bt_*.cpp/hpp</code> , <code>main.cpp</code>) y <code>src/third_party/</code> para la biblioteca de hash.
tests/	Casos de prueba automatizados (mínimo: verificación contra las instancias de referencia de la Sección 9) y utilidad de verificación de semilla.
resources/	diccionario.txt y cualquier archivo de datos de entrada necesario para reproducir los experimentos.
results/	Salidas generadas por la ejecución: tablas de tiempos (.csv), resultados de cada instancia (.txt), gráficas (.png).
report/	Informe.pdf (Sección 12).

El README debe permitir que el docente, sin ayuda adicional del equipo: (1) clone el repositorio; (2) compile con ``g++ -std=c++17 -O2 -o ada_p1 src/main.cpp src/*.cpp``; (3) ejecute el programa y reproduzca cada instancia de la Sección 9 con un único comando por módulo; (4) identifique a los integrantes del equipo y sus apellidos usados para calcular la semilla; (5) entienda, en un párrafo, la organización del proyecto.

12. Informe técnico

Documento entregado en `report/Informe.pdf`, con una extensión máxima de 12 páginas sin contar portada, estructurado exactamente así:

1. Portada (título, integrantes, fecha, curso).
2. Introducción (problema, motivación, estructura del documento).
3. Contexto del problema (síntesis de las Secciones 4 y 5 de este documento, en sus propias palabras).
4. Fundamentación teórica (síntesis de la Sección 6, conectada con su propia instancia del problema).
5. Modelamiento (espacio de búsqueda de cada módulo, según lo trabajado en la Sección 7).
6. Diseño algorítmico (decisiones de diseño de cada algoritmo).
7. Pseudocódigo (ambos módulos).
8. Implementación (decisiones relevantes de implementación; no se pide transcribir el código completo, solo los fragmentos que ilustren decisiones no triviales).
9. Análisis de complejidad (temporal y espacial, ambos módulos, teórico).
10. Casos de prueba (instancias usadas, según la Sección 9, incluyendo su semilla y el procedimiento de verificación).
11. Experimentación (tablas y gráficas de la Sección 8).
12. Resultados.
13. Análisis de resultados (contraste teórico vs. empírico).
14. Comparación algorítmica (fuerza bruta vs. diccionario; con poda vs. sin poda).
15. Conclusiones.
16. Referencias.
17. Uso de herramientas de IA (Sección 14).
18. Contribución individual de los integrantes (Sección 13).

13. Contribución individual

El informe debe incluir una sección "Contribución individual" donde cada integrante indique, de forma específica y verificable, qué actividades realizó: por ejemplo, diseño del algoritmo de un módulo, implementación, construcción de casos de prueba, experimentación, análisis de complejidad, análisis de resultados, redacción de una sección del informe. Esta distribución debe ser consistente con el historial de commits del repositorio: el docente podrá contrastar ambos. Una contribución declarada que no tenga respaldo verificable en el historial de commits podrá ser cuestionada durante la sustentación (Sección 16).

14. Uso de Inteligencia Artificial

Usos permitidos: TODOS

15. Entrega

- Fecha máxima de entrega: **lunes 17 de agosto de 2026**.
- Hora máxima: **11:59 p.m.**
- Medio de entrega: **InteractivaVirtual**.
- Un único integrante del equipo debe subir el enlace al repositorio de GitHub. Al momento de entregar el enlace, debe indicar explícitamente los nombres

completos de todos los integrantes del equipo, ordenados alfabéticamente por apellido.

- Si el mismo trabajo es subido por dos o más integrantes del mismo equipo, se descontarán **0.5 unidades** de la nota final de la práctica para todo el equipo, independientemente de quién haya realizado la entrega duplicada.

16. Sustentación oral

El docente podrá solicitar una sustentación oral individual o grupal a cualquier equipo, en el momento que lo considere pertinente, con o sin previo aviso. Durante la sustentación, el docente podrá formular preguntas sobre: funcionamiento de cada algoritmo, decisiones de diseño, pseudocódigo, análisis de complejidad, casos de prueba, resultados obtenidos, código fuente, decisiones de implementación y contribución individual declarada.

La incapacidad de un integrante para explicar con suficiencia la parte del trabajo que declaró haber desarrollado podrá afectar tanto su calificación individual como la calificación grupal de la práctica.

17. Observaciones y condiciones generales

- Esta práctica se desarrolla obligatoriamente en un repositorio de GitHub (Sección 11).
- El trabajo puede ser individual o en equipos de hasta 3 integrantes (Sección 13). No se admiten equipos de más de 3 integrantes.
- Queda prohibido usar este trabajo, en cualquiera de sus dos módulos, contra sistemas, cuentas, credenciales o infraestructura reales, propios o de terceros. El incumplimiento de esta condición se reporta como una falta grave, independiente de la calificación académica de la práctica.

- Todas las instancias de prueba deben ser reproducibles siguiendo exactamente los procedimientos de la Sección 9; los resultados que no puedan reproducirse a partir de la semilla declarada por el equipo no serán considerados válidos.
- Ante cualquier ambigüedad en el enunciado, debe consultarse al docente por el canal oficial del curso antes de la fecha de entrega; no se aceptan interpretaciones no consultadas como justificación de desviaciones respecto de lo solicitado.

RÚBRICA DE EVALUACIÓN

Escala: Excelente 4.6-5.0 · Bueno 4.1-4.5 · Satisfactorio 3.6-4.0 · Insuficiente 3.0-3.5 · Deficiente 0.1-2.9 · No realizado 0. Los pesos porcentuales suman exactamente 100% y reflejan que esta es una asignatura de Análisis de Algoritmos: el diseño algorítmico, el pseudocódigo, el análisis de complejidad y la comparación entre enfoques concentran más del 45% de la nota, frente a menos del 10% para código, reproducibilidad y repositorio combinados. La corrección funcional del programa (criterio 5) es necesaria pero, por sí sola, representa solo el 10% de la calificación.

Criterio	Peso	Excelente 4.6-5.0	Bueno 4.1-4.5	Satisfactorio 3.6-4.0	Insuficiente 3.0-3.5	Deficiente 0.1-2.9	No realizado 0
1. Comprensión conceptual	6%	Explica con precisión FB y BT, distingue completitud de poda y responde correctamente preguntas conceptuales en la sustentación.	Explica ambos paradigmas correctamente, con imprecisiones menores no estructurales.	Explica los paradigmas de forma básica, con al menos un concepto confundido (ej. poda cambia el orden de complejidad).	Definiciones incompletas o parcialmente incorrectas de uno de los dos paradigmas.	Confunde sistemáticamente Fuerza Bruta con Backtracking o con otros paradigmas.	Sección ausente.
2. Modelamiento del problema	6%	Espacio de búsqueda, estados y restricciones formalizados sin ambigüedad para ambos módulos, coherentes con la implementación.	Modelamiento correcto en ambos módulos con formalización menor incompleta.	Modelamiento correcto en un módulo; en el otro, informal o incompleto.	Modelamiento presente pero inconsistente con lo implementado.	Modelamiento ausente o gravemente incorrecto.	Sección ausente.
3. Diseño del algoritmo de Fuerza Bruta	8%	Enumeración sistemática demostrablemente completa y sin repeticiones; decisiones de diseño justificadas.	Enumeración correcta con justificación parcial de las decisiones de diseño.	Enumeración correcta pero sin justificación de decisiones de diseño.	Enumeración con omisiones o repeticiones detectadas en pruebas.	Enumeración incorrecta o incompleta de forma sistemática.	No implementado.
4. Diseño del algoritmo de Backtracking	8%	Función de factibilidad correcta y demostrablemente efectiva; poda documentada y justificada formalmente.	Poda correcta y funcional, justificación teórica incompleta.	Backtracking correcto pero con poda mínima o tardía (baja efectividad).	Implementación equivalente a enumeración exhaustiva sin poda real.	Backtracking incorrecto: omite soluciones válidas o acepta inválidas.	No implementado.
5. Correctitud de la implementación	10%	Ambos módulos compilan sin advertencias relevantes y	Correcto en ambos módulos, con advertencias	Correcto en un módulo; el otro presenta errores acotados.	Ambos módulos con errores que afectan	Uno o ambos módulos no compilan o producen	No implementado.

Criterio	Pe so	Excelente 4.6- 5.0	Bueno 4.1-4.5	Satisfactorio 3.6-4.0	Insuficiente 3.0-3.5	Deficiente 0.1- 2.9	No realizado 0
		producen resultados correctos en la instancia de referencia y en las instancias del equipo.	menores de compilación.		parcialmente los resultados.	resultados sistemáticamente incorrectos.	
6. Pseudocódigo	5%	Pseudocódigo de ambos algoritmos, correcto, completo y consistente con la notación del curso y con el código real.	Pseudocódigo correcto con inconsistencias menores frente al código.	Pseudocódigo presente pero incompleto en uno de los dos módulos.	Pseudocódigo ambiguo o con errores lógicos.	Pseudocódigo ausente en un módulo o gravemente incorrecto en ambos.	Sección ausente.
7. Análisis de complejidad	10 %	Complejidad temporal y espacial correcta y formalmente justificada para ambos módulos; discute mejor/peor caso en BT.	Análisis correcto con justificación formal incompleta.	Análisis correcto solo en notación final, sin derivación.	Análisis con errores en la cota reportada.	Análisis ausente o incorrecto en ambos módulos.	Sección ausente.
8. Casos de prueba	5%	Instancias de referencia y del equipo generadas exactamente según el procedimiento de la Sección 9, verificadas antes de usarse.	Instancias correctas con verificación parcial documentada.	Instancias correctas pero sin evidencia de verificación previa.	Instancias con errores en el cálculo de la semilla o los parámetros derivados.	Instancias inventadas o no derivadas del procedimiento exigido.	Sección ausente.
9. Experimentación	8%	Mínimo 5 configuraciones por módulo, tiempos medidos correctamente, al menos una gráfica por módulo, metodología reproducible.	Experimentación completa con una gráfica ausente o metodología parcialmente descrita.	Experimentación con menos de 5 configuraciones o sin gráfica.	Mediciones presentes pero incompletas o mal documentadas.	Experimentación simulada, incompleta o no ejecutada realmente.	Sección ausente.
10. Análisis de resultados	6%	Contraste explícito entre crecimiento	Contraste correcto con discusión de	Resultados descritos sin contraste	Análisis superficial, limitado a	Análisis ausente o incoherente	Sección ausente.

Criterio	Pe so	Excelente 4.6- 5.0	Bueno 4.1-4.5	Satisfactorio 3.6-4.0	Insuficiente 3.0-3.5	Deficiente 0.1- 2.9	No realizado 0
		teórico y empírico; identifica y explica el punto de crecimiento significativo con evidencia.	desviaciones incompleta.	explicito con la teoría.	repetir las tablas.	con los datos presentados.	
11. Comparación entre enfoques	6%	FB vs. diccionario y BT con poda vs. sin poda, ambas con métricas cuantitativas (tiempo, % de reducción, tasa de éxito) y discusión correcta de sus implicaciones.	Ambas comparaciones presentes, una de ellas con métricas incompletas.	Solo una de las dos comparaciones exigidas está completa.	Comparaciones presentes pero sin métricas cuantitativas.	Comparaciones ausentes o conceptualmente incorrectas.	Sección ausente.
12. Calidad del código	4%	Código modular, legible, con nombres significativos y organización clara entre módulos FB y BT.	Código correcto y razonablemente organizado, con documentación mínima.	Código funcional pero con baja modularidad o legibilidad.	Código desorganizado que dificulta su revisión.	Código ilegible o sin ninguna estructura reconocible.	No implementado.
13. Reproducibilidad	4%	Cualquier persona puede clonar, compilar y reproducir exactamente los resultados reportados siguiendo el README.	Reproducible con ajustes menores no documentados.	Reproducible parcialmente (un módulo reproducible, el otro no).	Reproducción requiere intervención significativa no documentada.	Resultados no reproducibles a partir del repositorio entregado.	Repositorio ausente o inaccesible.
14. Repositorio GitHub	4%	Estructura exacta de la Sección 11, historial de commits distribuido y consistente con la contribución declarada, README completo.	Estructura correcta con historial de commits parcialmente consistente.	Estructura correcta pero historial de commits concentrado en un integrante sin justificación.	Estructura incompleta o README insuficiente.	Repositorio desorganizado o sin historial de commits significativo.	Repositorio ausente.
15. Informe técnico	5%	Todas las 18 secciones de la Sección 12	Informe completo con carencias	Informe con 3 o más secciones incompletas.	Informe muy incompleto o que excede	Informe irrelevante o desconectado del	Informe ausente.

Criterio	Peso	Excelente 4.6-5.0	Bueno 4.1-4.5	Satisfactorio 3.6-4.0	Insuficiente 3.0-3.5	Deficiente 0.1-2.9	No realizado 0
		presentes, dentro del límite de extensión, con redacción técnica clara.	menores en 1-2 secciones.		notoriamente la extensión máxima sin justificación.	trabajo realizado.	
16. Referencias	2%	Todas las fuentes (bibliografía, biblioteca de hash, material del curso) citadas con formato consistente.	Referencias completas con formato inconsistente.	Referencias incompletas (falta alguna fuente usada).	Referencias mínimas, sin formato reconocible.	Ausencia de referencias pese a evidencia de fuentes externas usadas.	Sección ausente.
17. Uso ético de IA	2%	Declaración completa y específica (herramienta, fecha, propósito) coherente con la política de la Sección 14; sin evidencia de uso no declarado.	Declaración presente con nivel de detalle menor al exigido.	Declaración genérica ("se usó IA para dudas") sin especificar herramienta o propósito.	Declaración ausente pese a evidencia razonable de uso.	Evidencia de uso no declarado o de uso prohibido según la Sección 14.	No aplica / sección ausente sin evidencia de uso.
18. Contribución individual	1%	Declaración específica por integrante, verificable en el historial de commits, defendida correctamente en sustentación si se solicita.	Declaración específica con respaldo parcial en commits.	Declaración genérica ("todos hicimos todo") sin desagregación por actividad.	Declaración ausente o inconsistente con el historial de commits.	Evidencia de que un integrante no participó pese a estar declarado.	Sección ausente.